

1 Motivation

In this document, I will be outlining the scope of the computable semantic model by using a real example to motivate its features. The objective is to establish designs, reason about them and then use that to understand the scope of what the engine must be able to do. By doing this, the structure that needs to be implemented will reveal itself and there will principles will naturally arise that guide the implementation.

I will define basic computable semantic primitives and build the computable semantic model using that. The invariants specified by the CSP's will be enforced by reasoning about the design. For example, by going through this process, it will become clear that the validation mechanism will need to be able to find paths between nodes. In the same way, as I reason about the design I establish, I will make a list of features that I have to support.

This structured way of building this library will be more effective because it will force me to refine my thoughts and establish a structure that I will implement. It will also unambiguously define the scope of what is currently supported and how it can be expanded to support more features.

2 Design Abstraction Language

Currently, an AST library for the Design Abstraction Language (DAL) has been implemented with reserved keywords and structures, as well as the ability to specify custom commands. I am listing the features of the language in a general sense and then I will leverage them to meet the needs of the CSM.

2.1 Structure

The structural specification enables the construction of a block using braces. For example, the block can be used as follows to specify control flow logic for a behavior.

```
1 select {  
2     # Specify control flow logic  
3 }
```

2.2 Commands or Attributes

The command structure itself is specified as shown below. Each command is specified and in the bracket the arguments are provided. There can be keyword commands used by the design and then there are custom commands that reference more meaning.

```
1 # Built In  
2 design("Design Name")  
3  
4 # Custom Command  
5 command(  
6     number=123,  
7     string="String",  
8     arrayArg=[12,33,12],  
9     objArg={"a":1}  
10 )  
11  
12 # Reusing structure to specify attributes  
13 property(  
14     name="type"  
15     description="Data type of participant (list, object, str etc)"  
16     type="string"  
17     get="getType"  
18 )
```

2.3 Control Flow

The control flow structure itself is specified as specified below. Currently, only if statements are supposed but not else if or else blocks. I need to extend the library to support this.

```
1 # Currently supported  
2 if (boolean) {  
3  
4 }
```

This basic structure makes it possible to specify all the necessary attributes without making the AST implementation itself complicated. As I work through the reasoning and build the CSM, I will be using these structures specify the necessary meaning. If I run into features I can't support, I will return here and add to it.

2.4 Abstract Syntax Tree

The DAL script is parsed into an Abstract Syntax Tree (AST) that is then used by the validator. An example of a simple AST generated by a DAL script is provided below. In a full design specification, multiple dal files can be used to specify the behavior attributes, participant attributes etc.

2.4.1 Sample DAL Script

```
1 design("test")
2
3 behavior getName {
4   select {
5     goToBehavior(addBook)
6   }
7 }
8
9 behavior addBook {
10  #...
11  select {
12    goToBehavior(getName)
13  }
14 }
```

2.4.2 Sample AST

```
1 {
2   "type": "design",
3   "body": [
4     {
5       "type": "behavior",
6       "behaviorName": "getName",
7       "body": [
8         {
9           "type": "select",
10          "args": [],
11          "body": [
12            {
13              "type": "cmd",
14              "command": "goToBehavior",
15              "args": [
16                {
17                  "arg": null,
18                  "type": "name",
19                  "value": "addBook"
20                }
21              ]
22            }
23          ]
24        }
25      ]
26    },
27    {
28      "type": "behavior",
29      "behaviorName": "addBook",
30      "body": [
31        {
```

```

32         "type": "select",
33         "args": [],
34         "body": [
35             {
36                 "type": "cmd",
37                 "command": "goToBehavior",
38                 "args": [
39                     {
40                         "arg": null,
41                         "type": "name",
42                         "value": "getName"
43                     }
44                 ]
45             }
46         ]
47     }
48 ]
49 }
50 ],
51 "name": [
52     {
53         "arg": null,
54         "type": "string",
55         "value": "test"
56     }
57 ]
58 }

```

2.4.3 Modifying AST by injecting nodes

TODO — Implement feature to modify AST by injecting nodes

This is necessary because through the reasoning, the engine will make automatic modifications to the design by enforcing the designs own meaning. I am not talking about the actual types of changes yet, just the ability to add nodes.

3 Design - Library Manager

For the first design, I will be using a simple library manager because it exposes a lot of features while keeping the implementation simple. I will first informally discuss the meaning of the world and then I will formalize it using the design abstraction language. Then I will discuss the kinds of reasoning that can be done and identify the necessary features that the tool must have. Then I will describe how the meaning can be established using a semantic toolkit and automatically constructed as a closed semantic world.

3.1 Brief

In this section, I will informally describe the library manager and its purpose. The objective of the library manager is to do the following:

- Allow user to submit a book by providing its name and contents.
- Allow the book to be stored in the library.
- Allow user to checkout a book with a given name.
- Allow the book to be accessed efficiently when given the book name.

Building on these objectives, the requirements of the design can be derived. It is clear that there must be a mechanism to prompt and allow the user to select which action (submit or checkout). There must be a mechanism to allow the user to submit a book name for both submission and checkout.

Internally the library has to have a mechanism for storing the books and for accessing them when prompted by the user. Since the books must be accessed efficiently, i.e. not search through every book to find it, we can sort the books using the first letter of their name onto smaller shelves and only search those shelves. There are of course, more efficient ways to speed up the search but this simple method is ok for this design.

So now we already have a chance to take the first step towards using this framework. What I've described above begins to describe the world, there are objectives and the consequent requirements but in this framework, the meaning of the world is made completely unambiguous and computable, so the next step we take is to define the closed semantic world that establishes the meaning of the library as a computable model.

3.2 Eliminating Ambiguity

In the previous section, when describing the simple library manager, I was using many words whose meaning is established by the english language (Book, Library, User, Store etc.). It is because I know the meaning of these words that I was traditionally able to work towards the construction of a software system. The software development process was about establishing a design that realized unambiguous meaning and then implementing it in a programming language. However, this is also why it was possible to introduce bugs because it is an ambiguous process that has many translation steps while relying heavily on the skill of the developers to ensure that the system realizes its meaning and is internally consistent in its meaning.

In this framework, we establish the meaning of the world as a computable semantic model. The meaning of a book isn't established by the english language, it is established by the semantic model. This isn't a new concept, semantic models have been used to debug the behavior of a design in many solutions but by building the semantic model using computable semantic primitives, the meaning itself is made executable. This eliminates the many intermediate steps where bugs can be introduced and in the process, it also allows for automatic reasoning about the meaning of the world to eliminate known failures. So in this section, I am now going to establish the meaning of the library manager as a computable model. Then I will reason about the model and then I will describe how the engine can reason about the established structure.

3.2.1 The Nature of Meaning

A design can be described as a closed semantic world. A useful technique to establish the meaning of the world is to establish all the narratives that exist in this world. In this process, the meaning of the world itself becomes more and more unambiguous by identifying the participants in then narratives, reasoning about meaning in the narratives, the transformations made to the world state and the conditions under which those transformations will succeed.

Consider the following potential narrative:

A librarian accepts a name from the user

Using my understanding of sentences and meaning from the english language, I can say that the participants are *librarian*, *user*, *name*. The transformation is *accepts*, which seems to transfer ownership of the book name from the user to the librarian.

However, my interpretation of the meaning that is irrelevant, what is relevant is the meaning established by the computable semantic module because that it is what is realized by the software system. So this then requires the meaning of *librarian*, *user*, *name* and the transformation *accepts* to be established using the Design Abstraction Language and this allows the engine to reason about its meaning. Also, since the transformations are built from computable semantic primitives, the meaning becomes executable.

After establishing the computable model, a narrative could take the form shown below:

prompt for name from user then transfer name to librarian

In this example, user is prompted for a name. The meaning of *prompt* itself is established by the computable semantic model and in this case, it prompts the *user* for a *name* from the terminal. Then ownership of the *name* is transferred to the *librarian* participant. The *then* operator means that the transformations happen in a sequence.

The meaning itself can be built as composition of more meaning. This works the same way with language used by humans, the meaning of a lion includes all its attributes like its appearance, sound etc. So narratives can be established by composing computable meaning into higher level meaning.

This narrative can be described as a behavior of the design. The transformation modifies the world in an unambiguous way and produces an unambiguous world state after the transformation. After this narrative is exhibited, a subsequent narrative can be selected deterministically. In some cases, the meaning of the world state is reasoned about to select the next narrative and in other cases, it is directly selected. This chain of behaviors established through reasoning is what is referred to as the design.

In the next section, I will informally establish the narratives in the library manager and reason about them.

3.2.2 Narratives of the Library Manager

First, I will informally share the narratives of the library manager world and I will demonstrate how reasoning works using the informal narrative. This is an intermediate step that will eventually be eliminated because the narratives will be constructed using meaning established by the semantic toolkit.

The librarian presents the user with the option to add a book or get a book from the library.

If the user selects the option to add a book, the librarian accepts a book by asking the user for the name and content. Then the librarian reads the first letter of the books name to place the book on the relevant shelf. Then the user is once again asked which action they want to take.

If the user selects the option to get a book, the librarian asks the user for the book name. The librarian reads the first letter of the book name and accesses the book from the relevant shelf. After the librarian provides the user the book, the user is once again asked which action they want to take.

In the section above, an initial exploration of the narratives in the library manager world are established. From this narrative an example of reasoning about the meaning can be demonstrated. The librarian reads the first letter of the books name in two separate narratives with the expectation that there will be at least

one letter in the name. However, when accepting the book name, the librarian does not verify that the book name has at least one letter in it, by reasoning about the informal narrative provided above, it is clear that the world is inconsistent because it is not possible for it to realize its intentions in conditions that it allows.

Earlier, I mentioned that this step won't be necessary when the semantic toolkit becomes more mature because the software system will be automatically constructed from the established meaning. This is an important point because the computable semantic primitives initially populate the semantic toolkit and using them, more meanings can be constructed using narratives like the ones I showed above. So you can develop the entire software system entirely through narratives by composing more and more meaning until you reach the library manager.

I am intentionally building the design in this way to demonstrate how the meaning of the world is established because I think it is a useful to see. So in the next section, I will establish the meaning of the library manager itself as a computable semantic model by connecting all the behaviors to create the narrative flow by reasoning about the worlds meaning. Once this is done, since there is no ambiguity in the meaning of the world, an engine will be able to reason about it.

It will be evident from the structure that is created that you can automatically create the structure if the necessary narrative is specified. This will be demonstrated by creating narratives from the structure. So then I will demonstrate how you can automatically construct this model from computable semantic primitives up to the library manager using narratives.

3.3 Computable Semantic Model

In this section, I will manually build the meaning of the library manager as a computable semantic model. I will start by defining the behaviors of the design at a high level and the narratives of the design. Then I will define the meaning of participants and transformations for each of the behaviors. The transformations themselves will be constructed from computable semantic primitives.

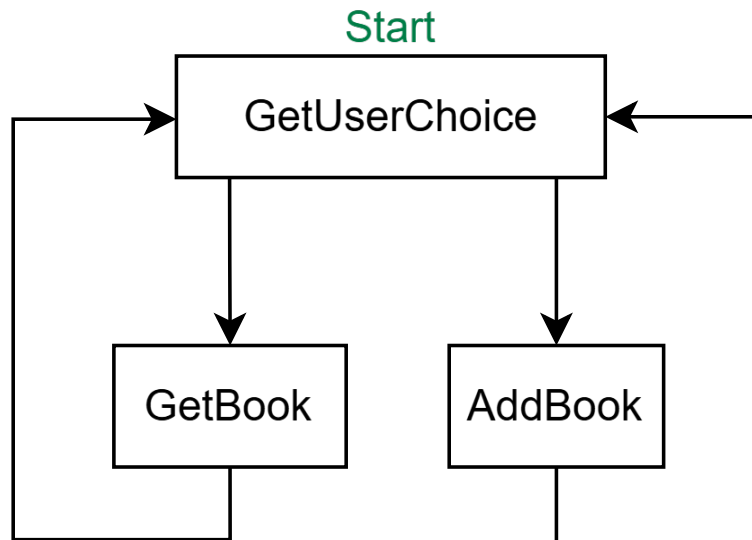


Figure 1: High Level Meaning of Library Manager

In Figure 1, the narratives at the highest level of meaning in the library manager are provided. The narrative begins at the behavior get user choice and then branches to get or add book based on the choice the user makes. The meaning of each of these behaviors is specified through the meaning that composes it. The following sections will establish the meaning of each of the behaviors and recursively continue until computable semantic primitives are reached.

More to come.

TODO// I will break down the meaning of each of the behaviors and recurse until I reach the primitives. Then I will show how you can construct meaningful narratives from the structure defined by the computable

semantic model I create through this process. Then it will become clear how the meaningful narratives can automatically create the computable semantic model and how the engine can reason about it. Using this, I will also present the scope of what the primitives need to establish.

3.4 Composing Meaning using Narratives

In the previous section, I demonstrated how you can build the meaning of the world from the highest level of meaning down to the primitives. I did this intentionally to demonstrate how meaning is constructed and how the design abstraction language is used to establish meaning using the computable semantic model. It is evident from the structure of the design that you can construct narratives using the established meaning and structure.

This means that the process I have established can now be reversed, you can compose meaning from the computable semantic primitives up to the library manager. As a result, you can build more meaning by establishing narratives constructed using the primitives and reason about it using the engine. Then you can build more meaning using the new meaning that was established by the narrative. This automates development by automatically constructing the world specified by the meaning of the narratives and reasoning about it.

The process I am describing will be somewhat familiar to developers because they traditionally used the mechanical primitives of a programming language to realize the meaning of the software system. In traditional approaches, the meaning remains in the developers head and in conjunction with the skill of the developer, the purpose of traditional software development approaches was an attempt to create a structured process that composes meaning of the software system using the programming language.

However, in this framework, the primitives of the programming language have meaning and they can be used to construct more meaning. Constructing a software system becomes a process of composing meaning by establishing the narratives which define the meaning and reasoning about it. Once the necessary meaning has been composed, the software system can be automatically synthesized from the meaning.

4 Design - Bubble Sort

The same process will be repeated for a bubble sort algorithm to demonstrate this process using a familiar design. It is not a particularly difficult design but that is what makes it interesting because it gives an opportunity to practice applying the principles and establishing its meaning.

5 Design - Distributed Rendering

The same process will be repeated for a distributed design, in this case, it is a distributed rendering application.

In this design, there is:

- One node:
 - that interfaces with the client and accepts jobs on one process
 - assigns the jobs to the rendering machines on another process and also monitors that the jobs are completed
- One database that stores the jobs and their states
- Four rendering machines

The rendering jobs themselves are 3d files submitted by the user and they are split into X mb chunks and rendered one chunk at a time until its done and then it is merged back together and shipped.

This will be a very neat design because it will force me to really confront some more corner cases in this framework while also exploring a distributed design. The only design that I've explored in this process that has concurrency in it is the multi threaded text translator, so this will be interesting.